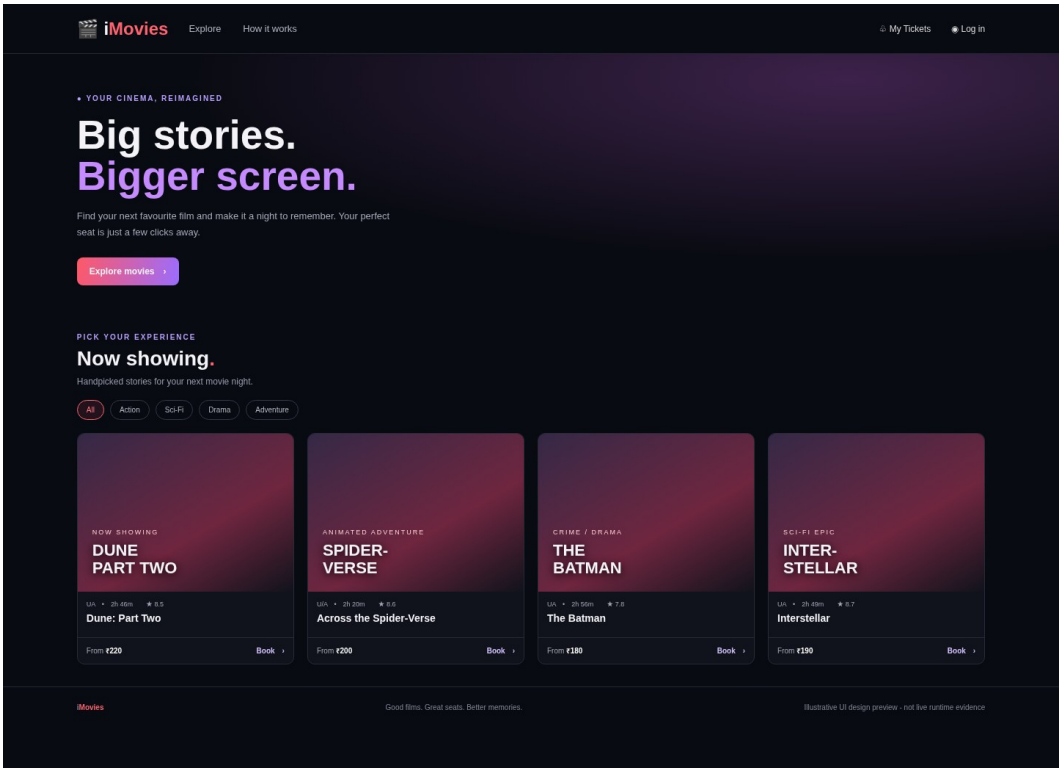
🎬 IMOVIES

MOVIE TICKET BOOKING MADE SIMPLE

Full Stack Developer - MERN | Intermediate Project | Version 2.0



Illustrative UI design preview (not a screenshot of a running instance).

Submission detail	Value
Student name	Amay Gupta
Enrollment number	EN24CS3010116
Project title	IMovies - Movie Ticket Booking Made Simple
Technology stack	MongoDB, Express.js, React.js, Node.js
Report version	2.0 - Expanded implementation source; hosted deployment and regression verification pending
Submission links	GitHub: https://github.com/AmayGuptaEN24CS3010116/IMovies | Live site: https://i-movies-amber.vercel.app/ | Demo video: add after recording

Document Guide

This report follows the supplied full-stack project template: project description and scenario, system requirements, architecture, project setup, backend and database implementation, frontend, execution, testing, limitations and final deliverables. It describes the expanded source package prepared for the next GitHub deployment.

Section	Contents
1	Project description, scenario, objectives and scope
2	System requirements
3	Project architecture, extended ER model, customer/admin user flow, features, roles, MVC/request handling, security and realtime updates
4	Project setup and configuration
5	Backend development and API endpoints
6	Database design, sample dataset and eight collection models
7	Frontend development
8	Execution guide
9	Test plan and output evidence
10	Limitations, future enhancements and references

Evidence status: the updated source package has passed JavaScript syntax checks and five automated helper unit tests in this environment. The current hosted website must be updated from the expanded source and the full browser regression checklist must be re-run before recording the final demo. Images in assets/previews/ are illustrative UI previews; only files in assets/screenshots/ captured by the student should be described as live output evidence.

1. Project Description

IMovies - Movie Ticket Booking Made Simple is a MERN-stack cinema-booking and management student application. It provides a movie catalogue with title, genre, language and city filters; MongoDB-backed theatre schedules; seat selection with Socket.IO refresh events; JWT user authentication; booking history and cancellation; persistent notifications; a simulated-payment ledger; and a role-protected admin dashboard for movies, theatres, shows and booking oversight. Payment is simulated and no real funds are collected.

1.1 Scenario

Moviegoers need to compare films, find a nearby cinema, choose a date and showtime, select available seats and keep their booking details accessible. IMovies demonstrates this process in one responsive application. Customers can browse/filter movies, open the available show schedule for a date and city, select a theatre and seats, confirm a demo booking and later review or cancel it. Administrators can maintain movies, theatres and schedules from a protected dashboard.

1.2 Problem Statement

The project demonstrates client-server separation, database modeling, authentication, role-based authorization, data validation, inventory protection, real-time UI updates and a persistent booking workflow. Its payment path is an explicit simulation for academic demonstrations, not a connected payment gateway.

1.3 Objectives

- Build a responsive movie catalogue with search and genre, language and city filters.
- Implement registration/login using bcrypt password hashing and JWT-protected API routes.
- Model users, movies, theatres, shows, seat reservations, bookings, demo payments and notifications in MongoDB/Mongoose.
- Let users select real scheduled shows and see current seat availability, including Socket.IO refresh events.
- Prevent duplicate reservations with a unique show-seat index; support booking cancellation and seat release.
- Provide a role-protected admin panel, paginated APIs, rate limits, unit tests and setup/deployment documentation.

1.4 Project Scope

Included in the expanded source	Not included / production limitation
Movie catalogue with title, genre, language and city filters	Real payment provider or real refunds; demo-only payment records
Registration/login, bcrypt, JWT and admin role checks	Email/SMS tickets and QR/PDF download
MongoDB-backed theatres/shows; show-specific seats and Socket.IO refresh	Full multi-document transaction/reconciliation and expiring seat-hold cleanup
Booking history, cancellation, seat release and persistent notifications	Production-grade admin audit, MFA and privileged account provisioning
Admin CRUD for movies, theatres and shows; booking overview; pagination and rate limits	Distributed rate limiting, monitoring, load testing and full e2e suite

2. System Requirements

2.1 Software Requirements

Software/tool	Purpose
Windows 10/11, macOS or Linux	Development environment
Node.js 18+ and npm	JavaScript runtime and package management
React + Vite	Client UI and local development server
Express.js	REST API routing and request handling
MongoDB Community Server or MongoDB Atlas	Persistent database
Mongoose	MongoDB schema modeling and queries
Visual Studio Code	Recommended code editor and terminal
Git and GitHub	Version control and project submission
Google Chrome / Firefox	Browser testing
Postman (optional)	Manual API request testing

2.2 Hardware Requirements

Component	Minimum recommendation	Purpose
Processor	Intel Core i5 (8th Gen) / AMD Ryzen 5 or similar	Development, IDE and browser use
RAM	8 GB minimum; 16 GB recommended	Run editor, server, database and browser together

Component	Minimum recommendation	Purpose
Storage	At least 1 GB free for source and packages; more for tools	Dependencies, database files and documentation
Display	1366 × 768 or higher	Review UI and work in code editor
Internet	Required for package installation and remote images/Atlas	Dependency downloads and remote services

3. Epic 1 - Project Architecture

The solution uses a React/Vite client, an Express REST API, a Socket.IO real-time channel, JWT/role middleware, and MongoDB via Mongoose. Public routes provide movies, active theatres, shows and seat availability. Authentication routes create/validate accounts. Protected booking, cancellation, payment-history and notification routes operate on the signed-in user. Admin routes are restricted to accounts whose stored role is `admin`. A unique seat-reservation index guards against duplicate show/seat claims.

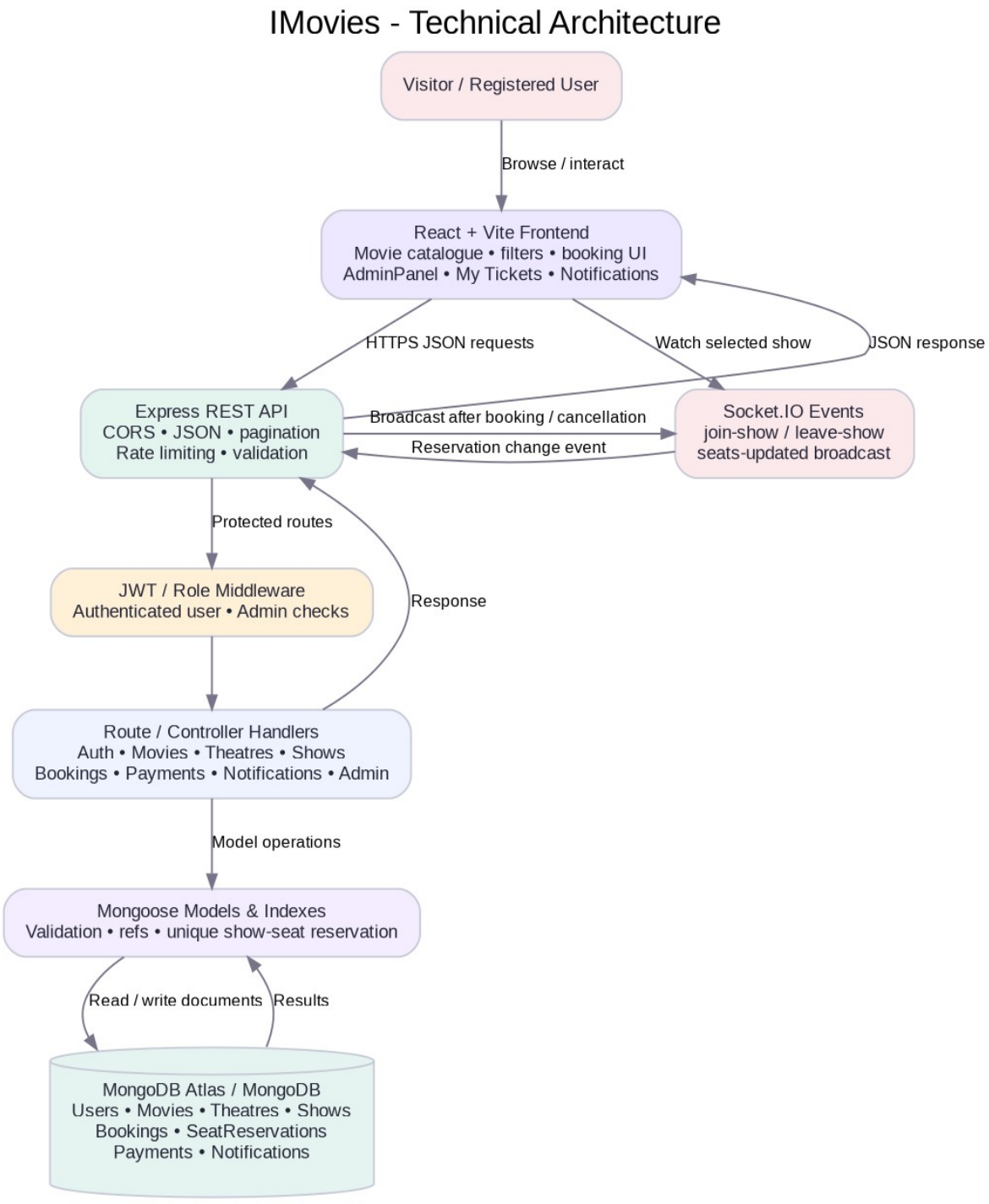


Figure 1. Technical architecture of the expanded IMovies application.

3.1 Architecture Components

Component	Responsibilities
React + Vite client	Renders movie catalogue, title/genre/language/city filters, show selector, realtime seat map, authentication, My Tickets, notifications and AdminPanel.

Component	Responsibilities
Express + Socket.IO API	REST endpoints for authentication, movies, theatres, shows, bookings, payments, notifications and administration; emits show-seat change events.
Authentication / authorization middleware	Verifies JWT, attaches current user and blocks admin routes unless stored role is `admin`.
Rate limits and validation	Limits API/auth/booking traffic; validates IDs, input fields, seat counts and access; centralized error handler standardizes responses.
Mongoose models and indexes	Define eight collection schemas, references, field rules, common-query indexes and unique show-seat reservations.
MongoDB	Persists users, catalogue, theatres, schedules, seat reservation records, bookings, simulated payments and notifications.

3.2 ER Diagram / Data Model

MongoDB stores the design as collections with ObjectId references. A user can own many bookings and notifications; a movie belongs to many shows and bookings; each show references one movie and theatre; a booking references a user and movie plus a show when booked through the scheduled-show flow. SeatReservation records protect a unique seat within a show. Payment entries are demo-only, and Notification records persist booking/cancellation updates.

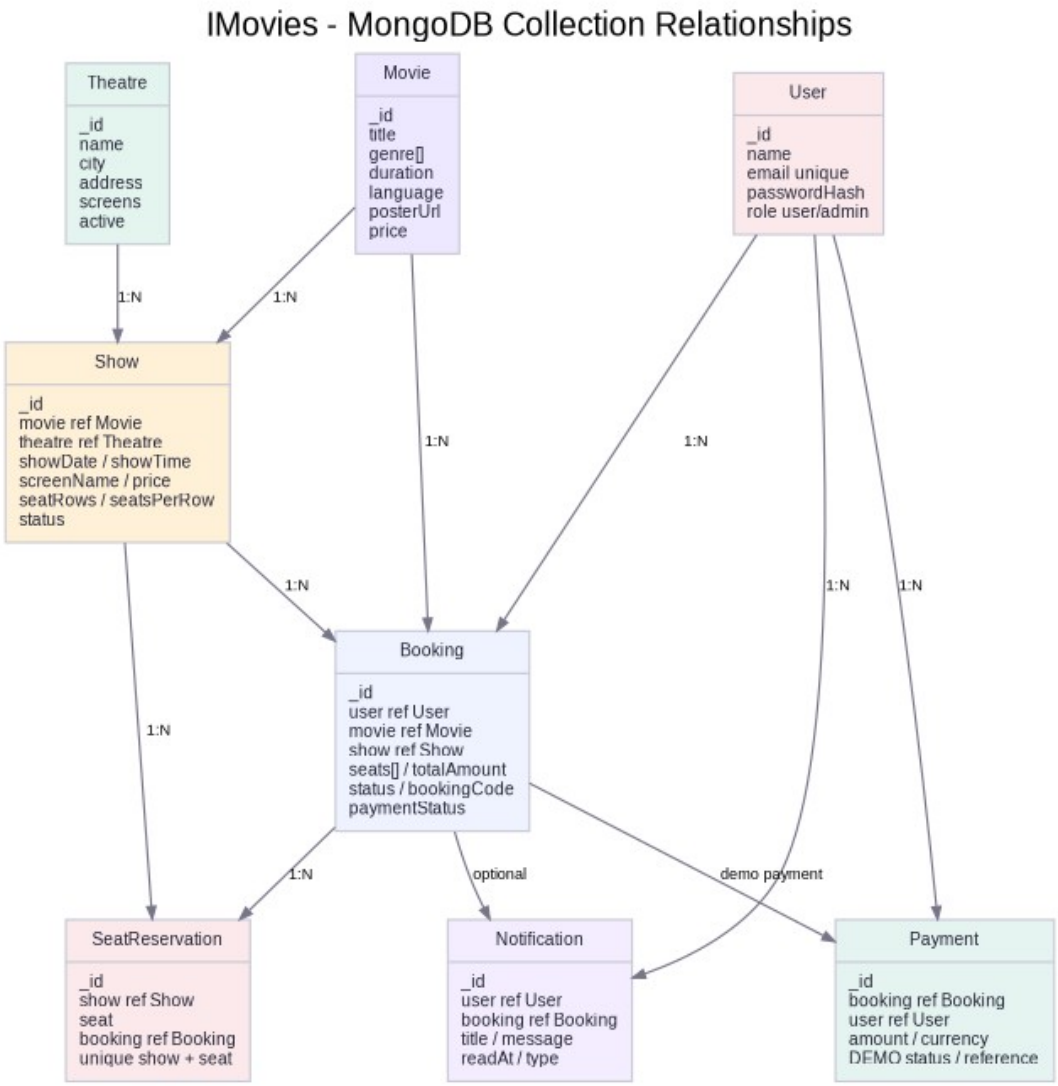


Figure 2. Expanded collection relationships: User, Movie, Theatre, Show, Booking, SeatReservation, Payment and Notification.

3.3 Collection Summary

Collection	Important fields	Relationship / constraints
User	name, email, passwordHash, role	Unique email; password hash only; role user/admin.
Movie	title, description, genre[], duration, rating, language, certificate, posterUrl, price	Catalogue record; price is default ticket price.
Theatre	name, city, address, screens, facilities, active	Active flag excludes a deactivated theatre from public choices.
Show	movie, theatre, showDate, showTime, screenName, language, price, seatRows, seatsPerRow, status	References Movie and Theatre; query indexes for schedule filtering.

Collection	Important fields	Relationship / constraints
SeatReservation	show, seat, booking, attemptId	Unique compound index `{ show, seat }` stops duplicate seat claims for a show.
Booking	user, movie, show, theatre, showDate, showTime, seats[], totalAmount, status, paymentStatus, bookingCode	Links user/movie/show; bookingCode is unique; cancellation releases its seat reservations.
Payment	booking, user, amount, currency, method, status, reference	DEMO method only; simulated success/refund; no external charge.
Notification	user, title, message, type, readAt, booking	Persistent user-specific booking/cancellation notices with unread tracking.

3.4 User Flow

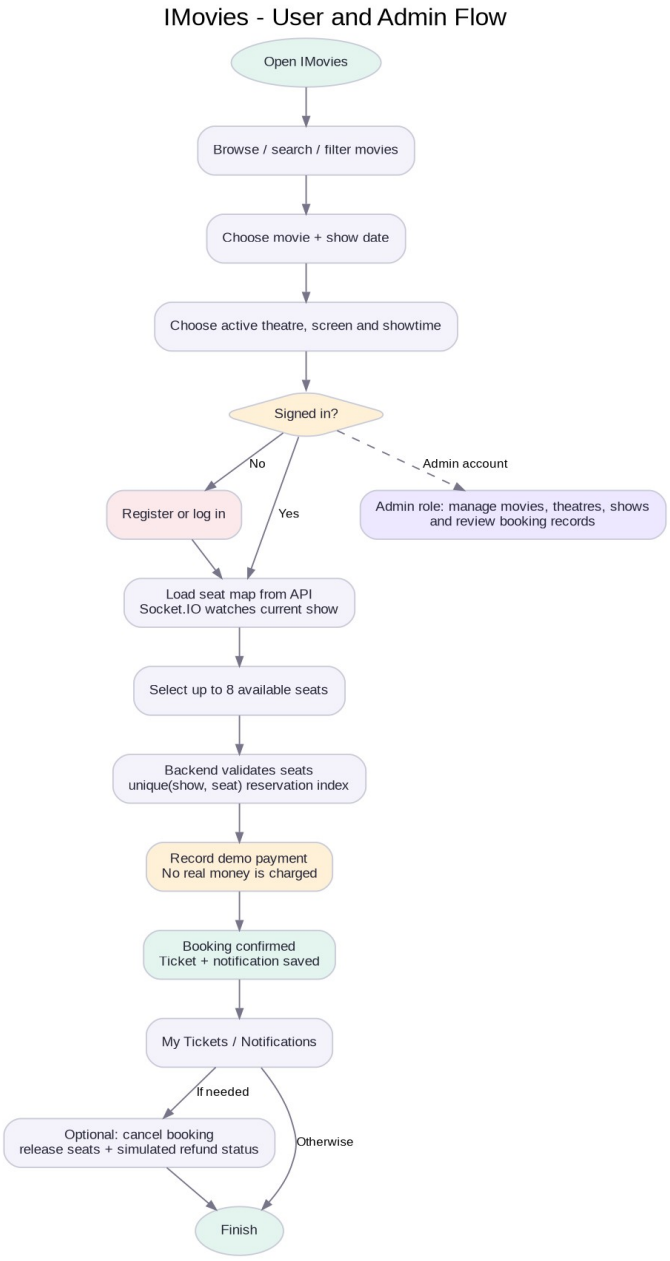


Figure 3. Customer booking and admin-management paths, including realtime seat refresh and cancellation.

3.5 Features

- Browse movie cards with genre, runtime, rating, language/certificate and starting ticket price.
- Search by title and filter by genre, language and city; scheduled shows can be filtered by movie, city, theatre and date.
- Register and sign in with email/password; passwords are hashed with bcryptjs and protected routes require JWT.
- Browse MongoDB-backed show records that include theatre, city, screen, date, time and price.
- Load the show-specific seat map from the API and receive Socket.IO updates after a booking or cancellation.
- Confirm a booking with server validation and unique show-seat reservation records; duplicate seat attempts return a conflict response.

- View saved bookings, booking codes, payment simulation status and notifications; cancel confirmed bookings to release reservations.
- Use an admin-only dashboard to create/read/update/delete movie records, create/update/deactivate theatres, schedule/edit/cancel shows and review bookings.

3.6 Roles and Responsibilities

Role	Responsibilities
Visitor	Browse and filter movies, theatres and scheduled shows; requires login to book.
Registered user	Manage own bookings, simulate checkout, view own booking/payment/notification records and cancel confirmed bookings.
Administrator	Access role-protected AdminPanel; create/update/delete movies, create/update/deactivate theatres, schedule/edit/cancel shows and inspect bookings/statistics.
Application/API	Validate requests, enforce JWT/role checks, reserve seats, write demo payments/notifications and broadcast seat status changes.
Developer / maintainer	Configure secrets, review Atlas access rules, run tests, manage deployments and protect privileged roles.

3.7 MVC / Request-Handling Pattern

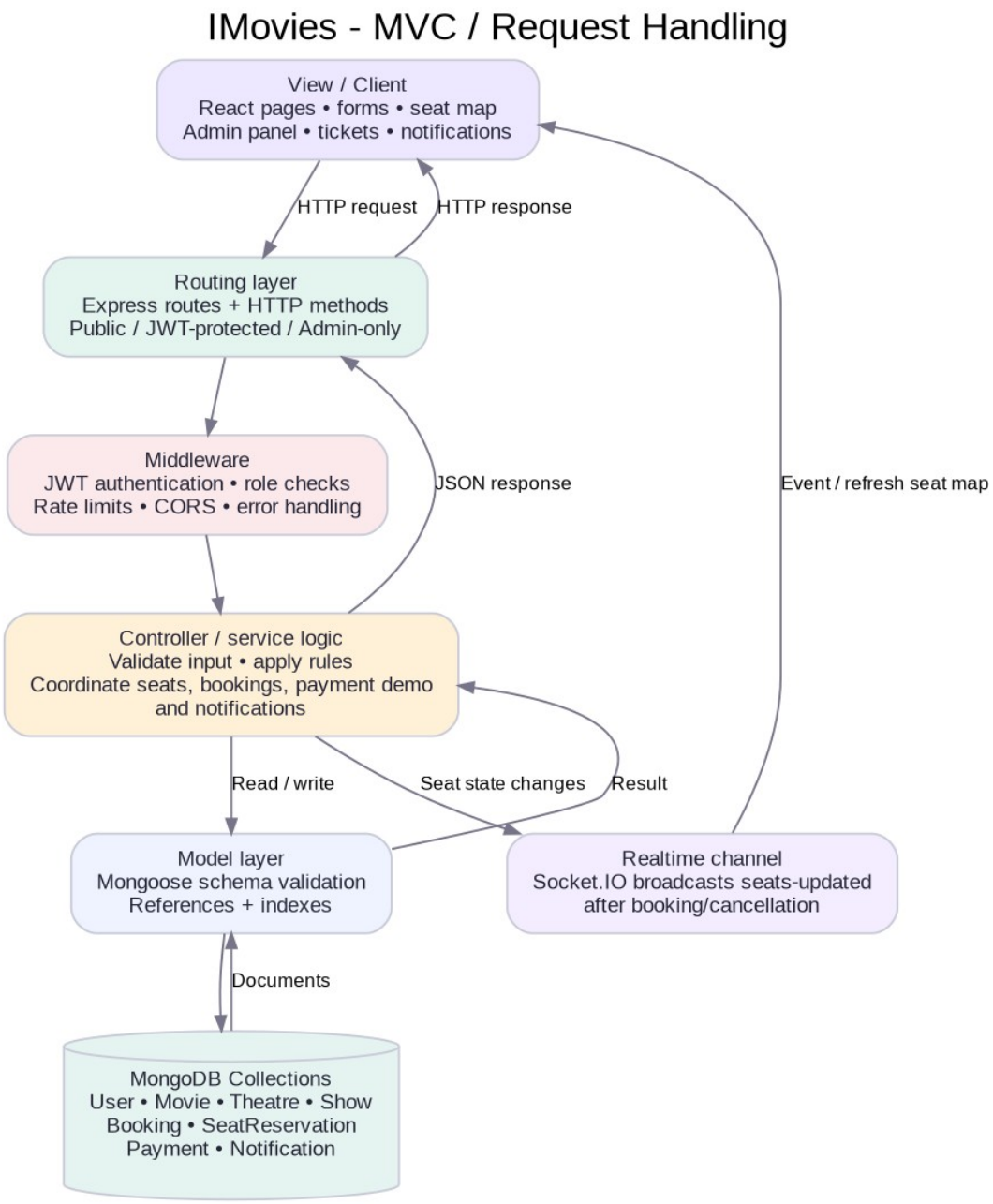


Figure 4. MVC/request-handling view including JWT middleware, admin authorization, Mongoose models and Socket.IO seat events.

Model layer: Mongoose schemas in `server/models/` define User, Movie, Theatre, Show, Booking, SeatReservation, Payment and Notification. Controller/request-handler logic: handlers in `server/routes/` validate requests, enforce user/admin permissions, coordinate database writes and persist notifications. View layer: React components under `client/src/` render the catalogue, booking modal, My Tickets, notifications and AdminPanel. Socket.IO broadcasts `seats-updated` to clients watching a show. The API returns JSON rather than server-rendered HTML.

3.8 Security and Validation Notes

- Passwords are hashed with bcryptjs before storage; plaintext passwords are not saved.

- JWT authentication protects booking, cancellation, payment-history and notification endpoints; admin routes also require the stored admin role.
- Requests validate required fields, seat counts, screen seat labels and object IDs; unique indexes prevent a duplicated seat claim for the same show.
- Never commit `.env` files or secrets. Use private Render environment variables and keep MongoDB credentials out of GitHub and screenshots.
- The show-seat unique index is the final guard against two requests reserving the same seat. However, booking, payment and notification writes span multiple collections and are not wrapped in one multi-document transaction; the project is a student demo rather than a production ticketing service. Production use also needs stronger admin provisioning, audit logging, monitoring and payment-provider verification.

4. Epic 2 - Project Setup and Configuration

4.1 Folder Structure

```
IMovies/
├── client/
│   ├── src/App.jsx
│   ├── src/AdminPanel.jsx
│   ├── src/main.jsx
│   ├── src/styles.css
│   ├── vite.config.js
│   └── package.json
├── server/
│   ├── models/ (User, Movie, Theatre, Show, SeatReservation, Booking, Payment, Notification)
│   ├── routes/ (auth, movies, theatres, shows, bookings, payments, notifications, admin)
│   ├── middleware/auth.js
│   ├── lib/seatUtils.js
│   ├── tests/seatUtils.test.js
│   ├── data/movies.json
│   ├── bootstrapData.js
│   └── index.js
├── assets/diagrams/ and assets/previews/
├── docs/ (report, tests, deploy, admin, demo)
├── README.md, LICENSE, .gitignore
```

4.2 Backend Setup

1. Install Node.js 18 or later and make sure `node --version` and `npm --version` work.
2. Start MongoDB locally or create an Atlas database and copy its connection URI.
3. Open a terminal in the `server` folder and copy `.env.example` to `.env`.
4. Set `MONGO_URI` and a private, long random `JWT_SECRET` in `server/.env`.
5. Run `npm install`, then `npm run seed` and `npm run dev`. The seed helper adds sample movies only if the movie collection is empty, and sample theatres/shows only if those collections are empty; it does not intentionally delete existing data.

```
cd server
# create .env from .env.example and edit values
npm install
npm run seed
npm run dev
```

4.3 Frontend Setup

6. Open a second terminal in the `client` folder and copy `.env.example` to `.env`.
7. Keep `VITE_API_URL=http://localhost:8000/api` for local backend use.
8. Run `npm install` and `npm run dev`, then open the Vite URL, usually `http://localhost:5173`.

```
cd client
# create .env from .env.example
npm install
npm run dev
```

4.4 Environment Variables

File / variable	Meaning	Safe handling
server/.env: PORT	Port for Express API (default 8000)	Local configuration
server/.env: MONGO_URI	MongoDB connection URI	Do not commit; may contain credentials
server/.env: JWT_SECRET	Secret used to sign/verify JWTs	Use a strong private value; do not commit
server/.env: CLIENT_URL	Allowed browser origin for local CORS	Allowed browser origin(s); exact Vercel production origin, no trailing slash. Localhost defaults are included in development.
client/.env: VITE_API_URL	Base URL for the API	Public client configuration only; never store secrets here

5. Epic 3 - Backend Development

The backend entry point is `server/index.js`. It connects to MongoDB, applies CORS and JSON middleware, applies global/auth/booking rate limits, mounts route modules and starts HTTP plus Socket.IO after MongoDB connects. On startup, `bootstrapData.js` fills missing sample data for an empty development/demo database.

5.1 API Endpoint Summary

Method	Endpoint	Purpose	Access
GET	/api/health	Health and realtime status	Public
GET	/api/movies	List/search/filter by genre, language and city; paginate	Public
GET	/api/movies/:id	Fetch movie details	Public
GET	/api/theatres	List active theatres; filter by city/name and paginate	Public
GET	/api/shows	Filter shows by movie/date/theatre/city and paginate	Public
GET	/api/shows/:id/seats	Return seat labels and occupied seats	Public
POST	/api/auth/register	Create account with hashed password and JWT	Public, rate-limited

Method	Endpoint	Purpose	Access
POST	/api/auth/login	Validate credentials and return role/JWT	Public, rate-limited
GET	/api/auth/me	Current profile	JWT
POST	/api/bookings	Reserve seat(s), confirm demo booking/payment, notification	JWT, rate-limited
GET	/api/bookings/my	Current user's bookings with pagination	JWT
PATCH	/api/bookings/:id/cancel	Cancel owned booking and release seat reservation	Owner/admin
GET	/api/notifications/my	List user's notifications and unread count	JWT
PATCH	/api/notifications/:id/read	Mark one notification read	JWT
PATCH	/api/notifications/read-all	Mark all notifications read	JWT
GET	/api/payments/my	List own payment-simulation records	JWT
GET	/api/admin/stats	Counts and demo totals	Admin
CRUD	/api/admin/movies	Create/read/update/delete movies	Admin
CRUD	/api/admin/theatres	Create/read/update/ deactivate theatres	Admin
CRUD	/api/admin/shows	Create/read/update/cancel scheduled shows	Admin
GET	/api/admin/bookings	Review booking records	Admin

5.2 Typical Booking Request

```
POST /api/bookings
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "showId": "<MongoDB show id>",
  "seats": ["A1", "A2"],
  "paymentMode": "demo"
}
```

The server validates the show and screen seat labels, reserves each show-seat pair, creates a booking, stores a simulated payment record and notification, and emits `seats-updated`. No real payment is processed.

The scheduled-show workflow receives `showId` and a seat array. The API loads the show and movie, validates that seat labels belong to the screen, creates unique SeatReservation records, rejects duplicate claims, saves a confirmed booking and a simulated Payment record, then writes a notification. It emits `seats-updated` to clients viewing the show. The response explicitly notes that no money was charged.

5.3 Error Responses

- 400 Bad Request: required fields missing, repeated seats or invalid seat labels.
- 401 Unauthorized: missing, invalid or expired JWT on a protected route.
- 404 Not Found: the requested movie, show, theatre or booking does not exist or is no longer available.
- 409 Conflict: duplicate email, duplicate seat reservation, or an operation blocked by linked records.
- 429 Too Many Requests: authentication, booking or API rate limit reached. 500 Internal Server Error: unexpected service/database failure.

6. Epic 4 - Database Development

6.1 MongoDB and Mongoose

The data layer uses eight collections: User, Movie, Theatre, Show, SeatReservation, Booking, Payment and Notification. Mongoose schemas define required fields, references, enum values, timestamps and indexes. User passwords are stored as passwordHash. Show records hold cinema scheduling and seat-layout metadata; SeatReservation has a unique `{ show, seat }` index; Payment records are labelled simulated and do not represent a real charge.

6.2 Seed Dataset

`server/data/movies.json` contains six sample movies with titles, descriptions, genres, durations, ratings, languages/certificates, poster URLs and prices. `server/seed.js` calls the non-destructive `ensureDemoData()` helper. If theatres/shows are absent, sample Bhopal theatres and a week of sample shows are created for demonstration.

Sample record	Genres	Duration	Demo price
Dune: Part Two	Sci-Fi, Adventure	166 min	₹220
Spider-Man: Across the Spider-Verse	Animation, Action	140 min	₹200
The Batman	Action, Crime	176 min	₹180
Interstellar	Sci-Fi, Drama	169 min	₹190
Jawan	Action, Thriller	169 min	₹180
12th Fail	Drama	147 min	₹160

6.3 Relationship Integrity and Limitations

Bookings reference a user and movie, and scheduled bookings also reference a show and theatre. The “my bookings” endpoint filters by authenticated user ID. SeatReservation has a unique index on show and seat; cancellation deletes the booking’s seat reservations so availability is restored. The multiple writes for booking/payment/notification are not wrapped in a single transaction, so the project should not be treated as production-grade inventory/payment software.

7. Epic 5 - Frontend Development

The frontend is a single-page React application bootstrapped with Vite. `App.jsx` manages catalogue filters, scheduled shows, realtime seat selection, authentication, bookings, cancellation and notifications. `AdminPanel.jsx` provides role-protected catalogue/theatre/show CRUD and a bookings overview. `styles.css` supplies the responsive dark cinema design, Socket.IO Client listens for show-seat events, and Lucide React provides icons.

7.1 Main Interface Areas

- Top navigation with Explore, How it works, My Tickets, notifications, login/account and a role-gated Admin action.
- Hero section introducing the booking experience and featured movie.
- Movie catalogue cards with title search, genre chips, language and city filters, ratings, metadata and starting prices.
- Booking modal with show date, scheduled theatre/showtime cards, screen layout, live seat map, total amount and clear demo-payment label.
- Login/register modal backed by the JWT authentication API.
- My Tickets shows booking code, theatre, date/time, seats, total, payment simulation status and cancellation control; notification modal displays persistent booking/cancellation updates.

7.2 UI Preview - Home

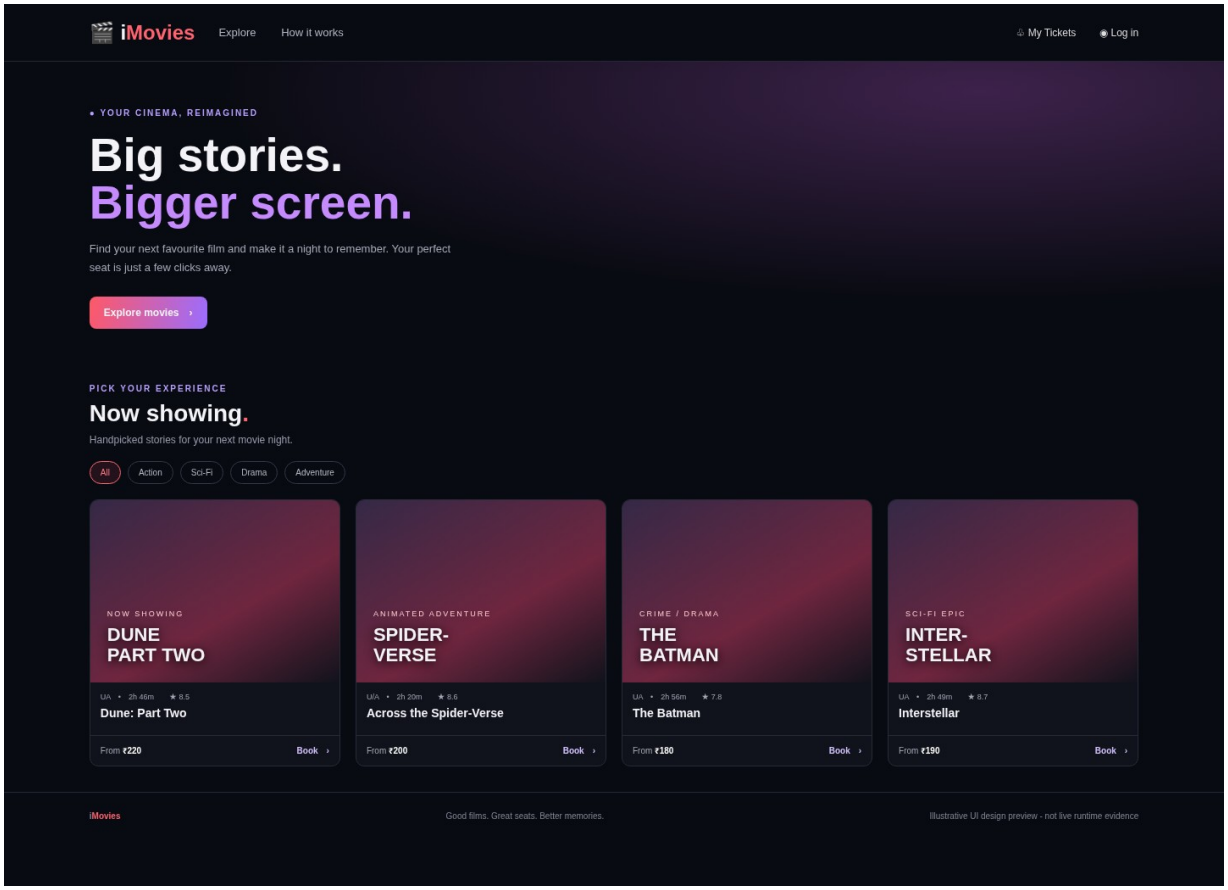


Figure 5. Illustrative home/catalogue preview. It is not a screenshot of the deployed React app.

7.3 UI Preview - Seat Selection

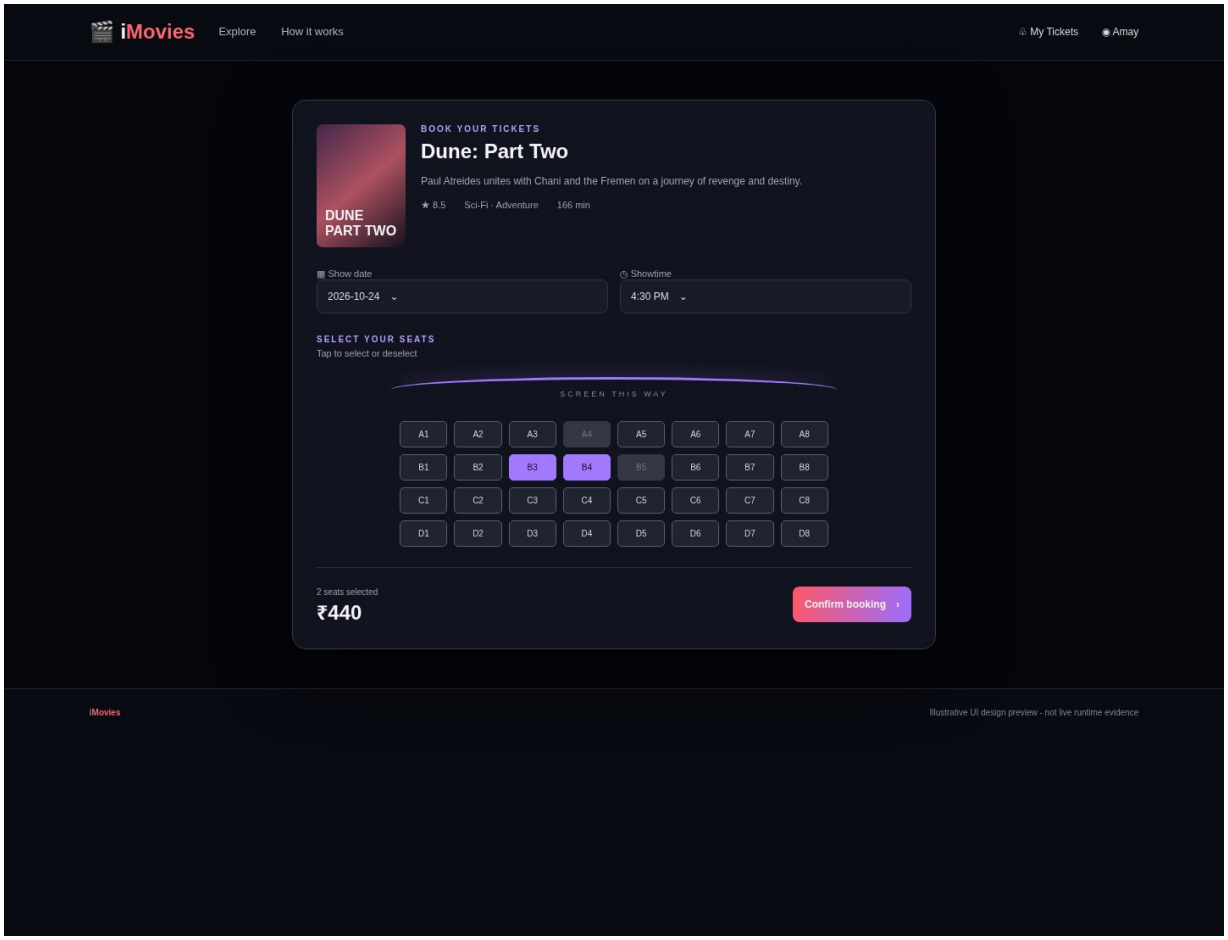


Figure 6. Illustrative booking/seat-selection preview. The deployed release loads seat availability from MongoDB-backed APIs.

7.4 UI Preview - My Tickets

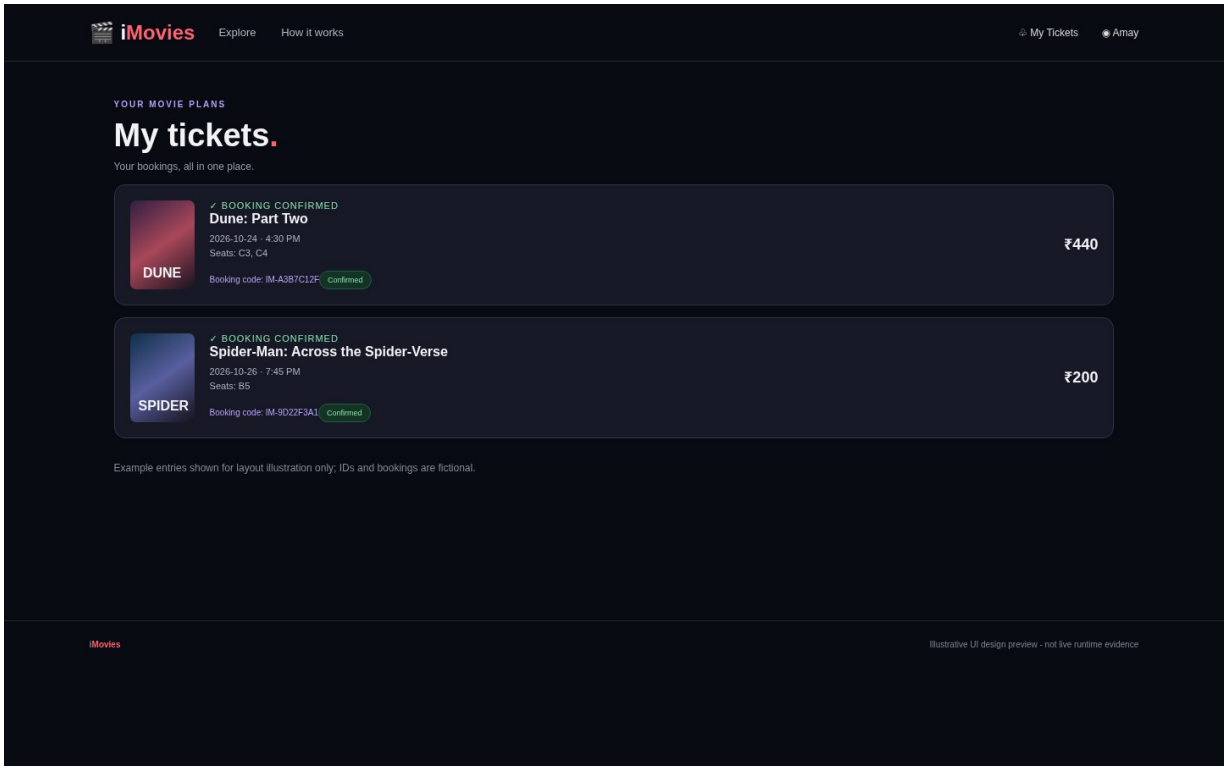


Figure 7. Illustrative My Tickets preview. Replace this image with a real captured ticket after final regression testing.

8. Epic 6 - Project Execution

8.1 Start the Application

9. Install Node.js 18 or later and ensure `node --version` and `npm --version` work.
10. Start MongoDB or configure an Atlas cluster, then in `server/` run `npm install`, `npm run seed` and `npm run dev`.
11. Check `http://localhost:8000/api/health` returns `status: ok` and the realtime flag.
12. In a second terminal, copy the client `.env.example` to `.env`, then run `npm install` and `npm run dev` inside `client/`.
13. Open the Vite development URL (usually `http://localhost:5173`) in a modern browser.

8.2 User Demonstration Steps

14. Browse/search/filter movies by title, genre, language and city.
15. Register or sign in, then choose a movie, future date and scheduled theatre/showtime.
16. Select available seats; if a seat becomes unavailable, refresh and choose another.
17. Confirm the simulated booking (no real payment); verify a booking record and notification are saved.
18. Open My Tickets, test cancellation on a confirmed test booking, and verify its seats become available again.

8.3 Troubleshooting

Symptom	Likely cause	Action
MongoDB connection failure	Service not started or URI incorrect	Start MongoDB; review `server/.env` and Atlas IP/network settings.
Frontend has no movies	Seed not run, API unavailable or browser cannot reach API	Run `npm run seed`; check backend terminal and `/api/health`.
CORS error	Client URL differs from server `CLIENT_URL`	Set the correct local origin in server `.env`, then restart backend.
401 on booking/My Tickets	User is logged out or token expired	Log in again and retry.
Poster image absent	External image URL or internet connection unavailable	Check connection; poster fallback placeholder may display.
Port already in use	Another local process occupies the port	Stop that process or choose a free port and update API URL as needed.

9. Testing and Output Evidence

The expanded source passed static JavaScript syntax checks and five Node.js native helper unit tests in the packaging environment. Network access did not permit a complete npm dependency installation/build here, and the expanded code has not yet been deployed and re-tested against the user's live services. Therefore this report does not mark live integration tests as passed. Complete `docs/TEST_PLAN.md` after committing the expanded files and waiting for Render/Vercel deployment.

9.1 Test Matrix

Test group	What to verify	Status in this package
Static source checks	Node syntax checks for server files and TypeScript parser syntax pass for JSX/config	Passed
Unit tests	Seat validation/layout and pagination helper tests	5 passed with Node built-in test runner
API availability	Health/show/theatre endpoints after upgraded Render deploy	Not yet re-tested after updated source upload
Database integration	Bootstrap sample theatres/shows and list existing movies	Must verify after updated source deploy
Authentication/security	Register/login/JWT, 401/403, rate limiting	Manual verification required after redeploy
Booking/seat controls	Book/cancel, duplicate seat conflict, show-seat release, Socket.IO updates	Manual multi-session verification required
Admin CRUD	Authorized user can manage movies/theatres/shows; standard user receives 403	Manual verification required after role setup
Frontend/responsive	Filters, show modal, My Tickets, notifications, AdminPanel	Browser regression required after Vercel redeploy
Payments	Demo payment entry exists and no real charge is made	Expected simulation design; verify in UI
Evidence	Screenshots and final demo URL	Add actual evidence after tests

9.2 Required Real Screenshots

After the expanded code is deployed and tested, capture genuine screenshots in `assets/screenshots/`: (1) home/catalogue; (2) title/genre/language/city filters; (3) selected show and theatre; (4) live seat map; (5) booking confirmation and My Tickets; (6) cancellation/released seats; (7) notifications; (8) admin dashboard and show management; and (9) public health response. Images in `assets/previews/` remain illustrative, not runtime evidence.

9.3 Demo Video

Use `docs/DEMO_SCRIPT.md` after the final regression checklist passes. Record the actual deployed website, upload the video to a course-approved share location, test that the link opens for a reviewer, and paste the real video URL into the submission worksheet. No demo video is fabricated or included in this package.

10. Limitations and Future Enhancements

10.1 Current Limitations

- No live payment-provider integration: the system records a clearly labelled simulated payment and no money is charged.
- The admin interface exists, but an authorized account must be deliberately promoted to the `admin` role by the trusted project owner for the classroom demo.
- The unique show-seat index prevents duplicate seat claims, but the entire booking/payment/notification workflow is not a single multi-document transaction and seat locks do not have expiry windows.
- Email/SMS ticket delivery, downloadable PDF/QR tickets and an external cinema inventory integration are not included.
- Rate limiting uses an in-process store and is not distributed across multiple server instances; production deployment requires more operational hardening, audit trails, monitoring and a reviewed secret-rotation process.
- Poster URLs depend on external image hosts and may expire or be blocked.
- The expanded source must be committed to GitHub and the latest Render/Vercel deployments must be verified before the final video is recorded. Full live tests are still required for the expanded release.

10.2 Future Enhancements

- Connect a real payment provider in test mode with signed webhook verification before considering any real payment use.
- Add a multi-document transaction/reconciliation strategy and expiring reservations/cleanup for abandoned checkouts.
- Add email/QR ticket delivery, user-facing ticket download and verified refund integration.
- Add administrator audit logs and a safer privileged account provisioning workflow.
- Add full API integration, browser end-to-end, load and accessibility tests plus CI.
- Replace in-memory rate-limit storage if horizontally scaling; add centralized logging and monitoring.
- Harden deployment configuration, narrow database network access and run regular dependency/security reviews.

11. Conclusion

The expanded IMovies source demonstrates a fuller MERN cinema workflow: movie discovery, theatre/show schedules, JWT authentication, show-specific seat reservations, realtime seat updates, cancellation, notifications, demo-payment records and role-protected administration. Five helper unit tests and static syntax checks passed during packaging. The upgraded source must be committed and deployed, the manual regression checklist completed, and genuine output screenshots captured before the final demo video and SkillWallet submission. Payments remain simulation-only.

12. References

- React. “Learn React.” https://react.dev/learn
- Vite. “Guide.” https://vite.dev/guide/
- Node.js. “Learn Node.js.” https://nodejs.org/en/learn
- Express.js. “Express documentation.” https://expressjs.com/
- MongoDB. “MongoDB Manual.” https://www.mongodb.com/docs/

- Mongoose. “Mongoose documentation.” https://mongoosejs.com/docs/
- JWT.io. “Introduction to JSON Web Tokens.” https://jwt.io/introduction
- npm. “bcryptjs package.” https://www.npmjs.com/package/bcryptjs
- Git. “Documentation.” https://git-scm.com/doc
- GitHub. “GitHub Docs.” https://docs.github.com/
- MDN Web Docs. “Web platform documentation.” https://developer.mozilla.org/
- Lucide. “Lucide React.” https://lucide.dev/guide/packages/lucide-react
- Socket.IO. “Socket.IO documentation.” https://socket.io/docs/v4/
- Express Rate Limit. “Documentation.” https://express-rate-limit.mintlify.app/overview
- Render. “Documentation.” https://render.com/docs
- Vercel. “Documentation.” https://vercel.com/docs

Appendix A - Submission Links

Repository and live-site links below have been filled from the project owner’s current deployment. Add the real demo-video URL only after recording and verifying the final expanded release.

Deliverable	Link / location
GitHub repository	https://github.com/AmayGuptaEN24CS3010116/IMovies
Live website	https://i-movies-amber.vercel.app/ (verify after expanded version redeploys)
Render health endpoint	https://imovies-api.onrender.com/api/health (verify after expanded version redeploys)
Demo video	Add actual shareable Google Drive/YouTube URL after recording final version.
Code Drive link	Only fill if SkillWallet separately requires it; do not invent a URL.
Report	docs/IMovies-Project-Report.pdf and editable DOCX
Source/dataset	client/, server/, server/data/movies.json
Architecture diagrams	assets/diagrams/
Runtime evidence	assets/screenshots/ - populate with authentic screenshots